```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.0;

contract MarksManagmtSys {

    // 1. STRUCTURE
    struct StudentStruct {
        uint ID;
        string fName;
        string lName;
        uint marks;
    }

    address public owner;
    uint public stdCount;

    // 2. ARRAY
    uint[] public studentIDs;

    // Mapping to store student records
    mapping(uint => StudentStruct) private stdRecords;

    event RecordAdded(
        uint ID,
        string fName,
        string lName,
        uint marks
    );

    // Owner restriction
    modifier onlyOwner() {
        require(msg.sender == owner, "Access denied: Only owner");
        _;
    }

    // Constructor
    constructor() {
        owner = msg.sender;
    }

    // Add new student record
    function addNewRecord(
        uint _ID,
        string memory _fName,
        string memory _lName,
        uint _marks
    ) public onlyOwner {

        require(_ID != 0, "Student ID cannot be zero");
        require(_marks <= 100, "Marks must be between 0 and 100");
        require(stdRecords[_ID].ID == 0, "Student ID already exists");

        stdCount++;

        studentIDs.push(_ID);

        stdRecords[_ID] = StudentStruct(
```

```
            _ID,
            _fName,
            _lName,
            _marks
        );

        emit RecordAdded(_ID, _fName, _lName, _marks);
    }

    // Get student record
    function getRecord(uint _ID)
        public
        view
        returns (StudentStruct memory)
    {
        require(stdRecords[_ID].ID != 0, "Record not found");

        return stdRecords[_ID];
    }

    // Get all student IDs
    function getAllStudentIDs()
        public
        view
        returns (uint[] memory)
    {
        return studentIDs;
    }

    // Get total students
    function getTotalStudents() public view returns (uint) {
        return stdCount;
    }

    // 3. FALLBACK FUNCTION
    fallback() external {
        revert("Function does not exist");
    }

    receive() external payable {
        revert("This contract does not accept Ether");
    }
}
```